How to Manipulate an Android Interface?

Mohammed Said BELAID
said.belaid@univ-usto.dz

14/03/2024
@USTO → FMI → Dept Info → L3 SI

Outline

1 Introduction

2 Activities

3 Fragments

4 Intents

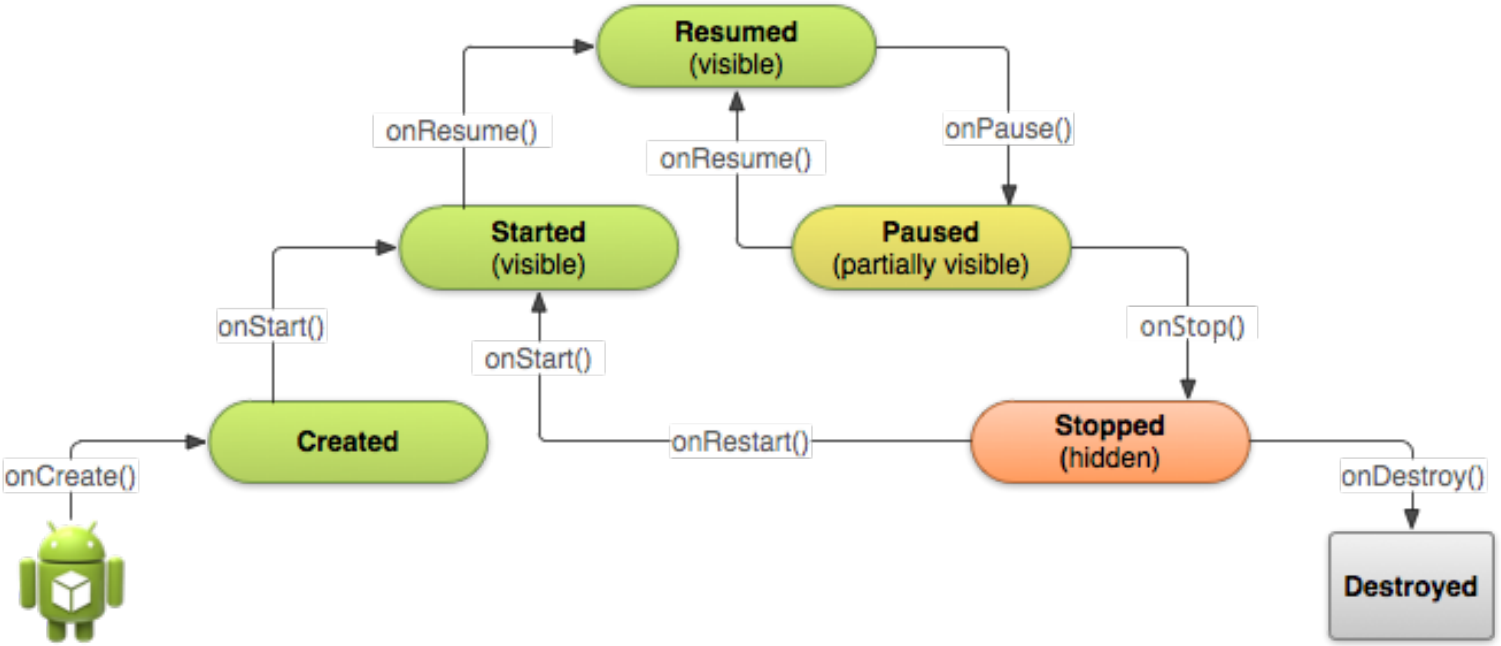
Objectives

- Android Activity Life cycle
- Access UI components via Kotlin/Java
- Handling events

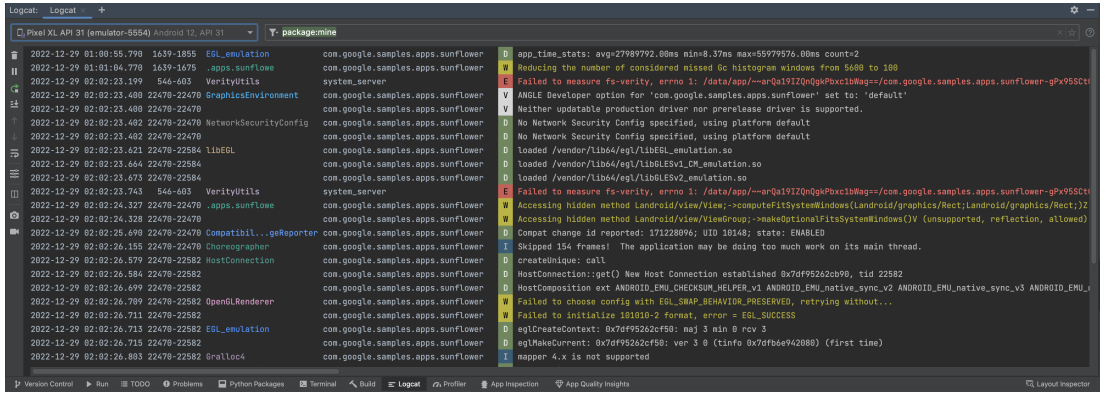
Android Application Components

- `Activities`: They dictate the UI and handle the user interaction to the smart phone screen.
- `Services`: They handle background processing associated with an application.
- `Broadcast Receivers`: They handle communication between Android OS and applications.
- `Content Providers`: They handle data and database management issues.

Activity Life Cycle

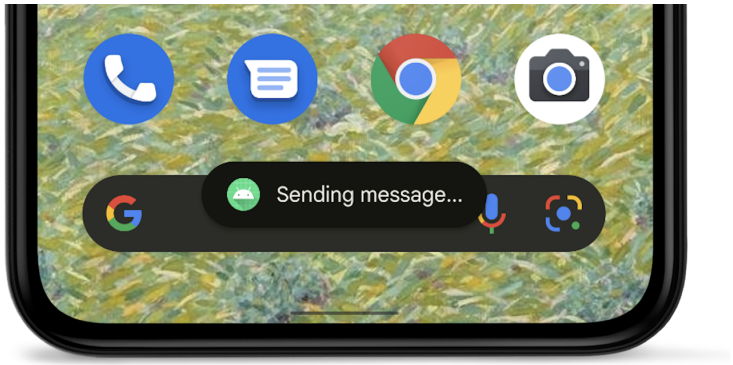


Logs



- helps debugging android apps by displaying logs from devices in real time
- Example: `Log.e("Test", "this is an error")`
- `Log.(i, e, d, v ...)`

Toasts



- Display a feedback to users in a small temporary popup

Reference an element on the View

- `findViewById:` we can get UI element from the layout by using its ~id.
- `ButterKnife`: (deprecated)
- `DataBiding`: reference state vars from the view itself [1]
- `ViewBinding`: generate references automatically in the Activity [2]
- `States` in Jetpack Compose: (TBD)

[1] https://developer.android.com/topic/libraries/data-binding
[2] https://developer.android.com/topic/libraries/view-binding

findViewById

```
class MainActivity : AppCompatActivity() {
  override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    val textView = findViewById<TextView>(R.id.textView)
  }
}
```

Update Interface content dynamically

- Once an UI element is referenced from Java or Kotlin, we can update all its attributes
  - Change the content
  - Colors
  - Visibility
  - ...

```
val textView = findViewById<TextView>(R.id.textView)
textView.text = "Updated text"
```

DataBinding

```
<layout ...>
    <data>
      <variable
  name="viewmodel"
  type="com.myapp.data.ViewModel" />
    </data>
    <ConstraintLayout... >
      ...
      <TextView android:text="@{viewmodel.userName}" />
      ...
    </ConstraintLayout>
</layout>
```

View Binding

my_activity.xml

```
<LinearLayout ... >
  ...
  <TextView android:id="@+id/
      name" />
  ...
</LinearLayout>
```

```
override fun onCreate(
    savedInstanceState: Bundle?)
    {
    super.onCreate(
        savedInstanceState)
    binding = MyActivityBinding.
        inflate(layoutInflater)
    val view = binding.root
    binding.name.text =
        viewModel.name
    setContentView(view)
}
```

Handling user input

- We can also get the user inputs in the forms UI elements, EditText for Example

```
editText.getText()
```

Implementing event listener

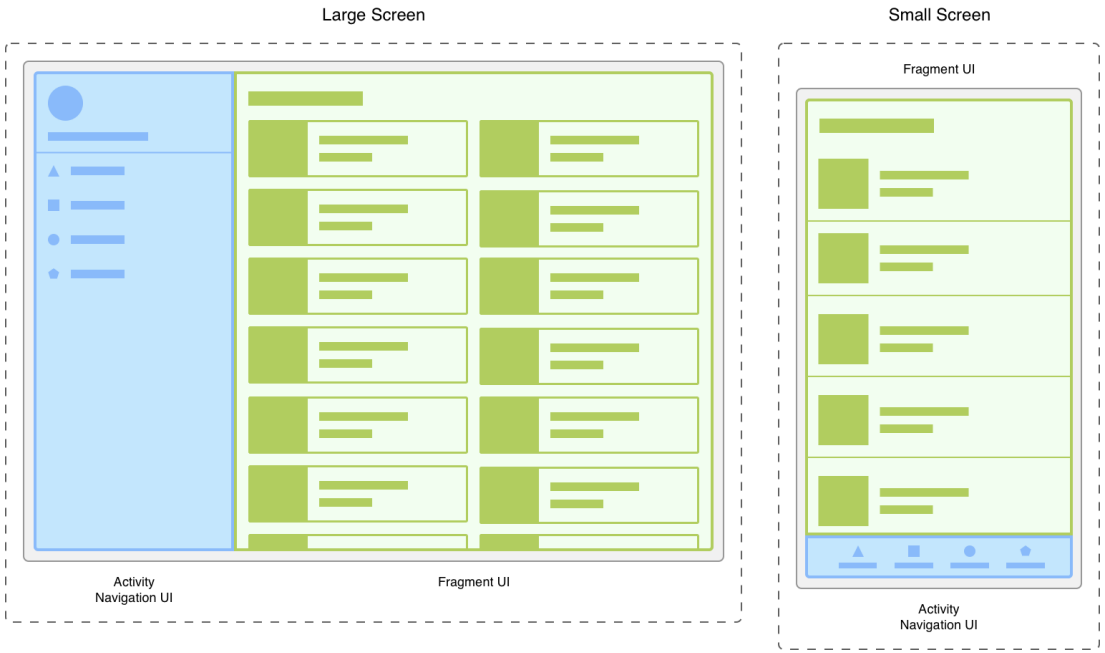
- The most common event is the click, the listener can be handled in:
  - The layout, with `onClick` attribute that call a function in the Activity class

```
<Button
...
onClick="clickMe"/>
```

  - The activity, by setting the On Click Listener: `setOnClickListener`

```
button.setOnClickListner { }
```

Fragments



- a reusable portion of app's UI.
- defines and manages its own layout
- has its own lifecycle
- handle its own user interactions

Intents (Next Course)